Lab 5: Writing a Dynamic Storage Allocator

Assigned: Wednesday, March 4, 2026
Due Date: *Friday, March 13, 2026 at 11:59 p.m.*
Close Date: *Friday, March 13, 2026 at 11:59 p.m* **- NO LATE DAYS -**
Video(s): (1) `block_info` struct [YouTube] (https://youtu.be/-Riqk0rlsP8), (2) Dynamic memory blocks [YouTube] (https://youtu.be/VSWqd83T6ew), (3) Macros in C [YouTube] (https://youtu.be/ycHTELCpQDs), (4) Provided macros [YouTube] (https://youtu.be/qyXGssUP0AU), (5) Trace file debugging [YouTube] (https://youtu.be/7vTW_VwE22Q), and (6) GDB `backtrace` demo [YouTube] (https://youtu.be/nKPsBQbAros)

Overview

Learning Objectives:

- Implement a memory allocator using an explicit free list.
- Examine how algorithm choice impacts tradeoffs between utilization and throughput.
- Read and modify a substantial C program.
- Improve your C programming skills including gaining more experience with structs, pointers, macros, and debugging.

In this lab, you will be writing a dynamic storage allocator for C programs, *i.e.*, your own version of the `malloc` and `free` routines. This is a classic implementation problem with many interesting algorithms and opportunities to put several of the skills you have learned in this course to good use. However, be warned that it is quite involved, so *start early!*

Code for this lab

Browser: `lab5.tar.gz` (lab5.tar.gz)
Terminal: `wget https://courses.cs.washington.edu/courses/cse351/26wi/labs/lab5.tar.gz`

Unzip: Running `tar xzvf lab5.tar.gz` from the terminal will extract the lab files to a directory called `lab5`.

Lab 5 Instructions

Lab Format

The only file you will modify and turn in is `mm.c` (unless you decide to do extra credit). You may find the short `README` file useful to read.

Your dynamic storage allocator will consist of the following three functions (and several helper functions), which are declared in `mm.h` and defined in `mm.c`:

- `int mm_init(void);`
- `void* mm_malloc(size_t size);`
- `void mm_free(void* ptr);`

The `mm.c` file we have given you partially implements an allocator using an explicit free list. Your job is to complete this implementation by filling out `mm_malloc` and `mm_free`. The three main memory management functions should work as follows:

- `mm_init` (provided): Before calling `mm_malloc` or `mm_free`, the application program (*i.e.*, the trace-driven driver program that you will use to evaluate your implementation) calls `mm_init` to perform any necessary initializations, such as allocating the initial heap area. The return value is -1 if there was a problem in performing the initialization, 0 otherwise.
- `mm_malloc`: The `mm_malloc` routine returns a pointer to an allocated block payload of at least `size` bytes. (`size_t` is a type for describing sizes; it's an unsigned integer that can represent a size spanning all of memory, so on x86_64 it is a 64-bit unsigned value.) The entire allocated block should lie within the heap region and should not overlap with any other allocated block.
- `mm_free`: The `mm_free` routine frees the block pointed to by `ptr`. It returns nothing. This routine is guaranteed to work only when the passed pointer (`ptr`) was returned by an earlier call to `mm_malloc` and has not yet been freed. These semantics match the semantics of the corresponding malloc and free routines in libc. Type `man malloc` in the shell for complete documentation.

We will compare your implementation to the version of malloc supplied in the standard C library (libc). Since the libc malloc always returns payload pointers that are aligned to 8 bytes, your malloc implementation should do likewise and always return 8-byte aligned pointers.

Provided Code

We define a `block_info` struct designed to be used as a node in a doubly-linked explicit free list, and the following functions for manipulating free lists:

- `block_info* search_free_list(size_t req_size)` : returns a block of at least the requested size if one exists (and `NULL` otherwise).
- `void insert_free_block(block_info* free_block)` : inserts the given block in the free list in a LIFO manner.
- `void remove_free_block(block_info* free_block)` : removes the given block from the free list.

In addition, we implement `mm_init` and provide two helper functions implementing important parts of the allocator:

- `void request_more_space(size_t req_size)` : enlarges the heap by `req_size` bytes (if enough memory is available on the machine to do so).
- `void coalesce_free_block(block_info* old_block)` : coalesces any other free blocks adjacent in memory to `old_block` into a single new large block and updates the free list accordingly.

Finally, we use a number of C static inline functions and preprocessor macros to extract common pieces of code (constants, annoying casts/pointer manipulation) that might be prone to error. Each is documented in the code. You are welcome to create your own macros as well, though the ones already included in `mm.c` are the only ones we used in our sample solution, so it's possible without more. For more info, check the Macros section of the GCC manual (http://gcc.gnu.org/onlinedocs/cpp/Macros.html).

- `FREE_LIST_HEAD` : returns a pointer to the first block in the free list (the head of the free list).
- `UNSCALED_POINTER_ADD` and `UNSCALED_POINTER_SUB` : useful for calculating pointers without worrying about the size of `struct block_info` .
- Other short utilities for extracting the size field and determining block size.

Additionally, for debugging purposes, you may want to print the contents of the heap. This can be accomplished with the provided `examine_heap()` function.

Memory System

The `memlib.c` package simulates the memory system for your dynamic memory allocator. In your allocator, you can call the following functions (if you use the provided code for an explicit free list, most uses of the memory system calls are already covered).

- `void* mem_sbrk(int incr)` : Expands the heap by `incr` bytes, where `incr` is a positive nonzero integer and returns a pointer to the first byte of the newly allocated heap area. The

semantics are identical to the Unix `sbrk` function, except that `mem_sbrk` accepts only a positive nonzero integer argument. (Run `man sbrk` if you want to learn more about what this does in Unix.)

- `void* mem_heap_lo()` : Returns a pointer to the first byte in the heap.
- `void* mem_heap_hi()` : Returns a pointer to the last byte in the heap.
- `size_t mem_heapsize()` : Returns the current size of the heap in bytes.
- `size_t mem_pagesize()` : Returns the system's page size in bytes (4K on Linux systems).

The Trace-driven Driver Program

The driver program `mdriver.c` in the `lab5.tar.gz` distribution tests your `mm.c` package for correctness, space utilization, and throughput. Use the command `make` to generate the driver code and run it with the command `./mdriver -V` (the `-V` flag displays helpful summary information as described below).

The driver program is controlled by a set of *trace files* that it will expect to find in a subdirectory called `traces` . The .tar.gz file provided to you should unpack into a directory structure that places the `traces` subdirectory in the correct location relative to the driver. (If you want to move the trace files around, you can update the TRACEDIR path in `config.h`).

Each trace file contains a sequence of allocate and free directions that instruct the driver to call your `mm_malloc` and `mm_free` routines in some sequence. The driver and the trace files are the same ones we will use when we grade your submitted `mm.c` file. Trace files are structured in the following manner:

```
20000       # suggested heap size (unused)
2           # number of ids -- in this case, 0-1
4           # number of alloc + free operations
1           # weight for this tracefile (unused)
a 0 2040    # alloc block "0" with payload size 2040
a 1 2040    # alloc block "1" with payload size 2040
f 1         # free block "1"
f 0         # free block "0"
```

The `mdriver` executable accepts the following command line arguments:

- `-t <tracedir>` : Look for the default trace files in directory `tracedir` instead of the default directory defined in `config.h` .
- `-f <tracefile>` : Use one particular `tracefile` for testing instead of the default set of tracefiles.
- `-h` : Print a summary of the command line arguments.
- `-l` : Run and measure `libc` malloc in addition to the student's malloc package.

- `-v` : Verbose output. Print a performance breakdown for each tracefile in a compact table.
- `-V` : More verbose output. Prints additional diagnostic information as each trace file is processed. Useful during debugging for determining which trace file is causing your malloc package to fail.

Programming Rules

- You should not change any of the interfaces in `mm.c` (*e.g.*, names of functions, number and type of parameters, etc.).
- You should not invoke any memory-management related library calls or system calls. This means you cannot use `malloc` , `calloc` , `free` , `realloc` , `sbrk` , `brk` or any variants of these calls in your code. (You may use all the functions in `memlib.c` , of course.)
- You are not allowed to define any global or `static` compound data structures such as arrays, structs, trees, or lists in your `mm.c` program. You *are* allowed to declare global scalar variables such as integers, floats, and pointers in `mm.c` , but try to keep these to a minimum. (It is possible to complete the implementation of the explicit free list without adding any global variables.)
- For consistency with the `malloc` implementation in `libc` , which returns blocks aligned on 8-byte boundaries, your allocator must always return pointers that are aligned to 8-byte boundaries. The driver will enforce this requirement for you.

Evaluation

Your grade will be calculated (as a percentage) out of a total of 56 points as follows:

- **Correctness (36 points):** You will receive 4 points for each test performed by the driver program (9 tests total) that your solution passes.
- **Style (6 points):** This lab requires a significant amount of C programming, so we will be expecting you to use proper style. That being said, this isn't a programming course, so what we're asking for won't be anything unreasonable. This part of your grade should be relatively stress-free, so we've explicitly stated what we will be looking for:
  - Comments: Since some of the pointer manipulation inherent to allocators can be confusing, we expect to see short inline comments explaining your code, as well as function headers for any helper functions you may define. (These will also help us give you partial credit if you have a partially working implementation.)
  - Macros and Static Inline Functions: You should use the provided macros and static inline functions when appropriate. You should not reimplement code or hard-code constants when there exists a macro or static inline function that does this for you.
  - Readability: Your TAs will be reading through your implementation while grading, so write your code with that in mind. Some things to think about include using proper indentation,

descriptive variable names, etc. If you are comfortable with general 143 style, that should be sufficient.

- **Performance (5 points):** Performance represents a small portion of your grade. We are most concerned about the correctness of your implementation. For the most part, a correct implementation will yield reasonable performance. Two performance metrics will be used to evaluate your solution:
  - Space utilization: The peak ratio between the aggregate amount of memory used by the driver (*i.e.*, allocated via `mm_malloc` but not yet freed via `mm_free`) and the size of the heap used by your allocator. The optimal ratio is 1, although in practice we will not be able to achieve that ratio. You should find good policies to minimize fragmentation in order to make this ratio as close as possible to the optimal.
  - Throughput: The average number of operations completed per second.

  The driver program summarizes the performance of your allocator by computing a **performance index**, P, which is a weighted sum of the space utilization and throughput: `P = 0.6U + 0.4 min (1, T/`T_{libc}`)`, where U is your space utilization, T is your throughput, and T_{libc} is the estimated throughput of `libc` malloc on your system on the default traces.

  - The performance index favors space utilization over throughput. You will receive `5*(P+0.1)` points, rounded **up** to the closest whole point. For example, a solution with a performance index of 0.63 or 63% will receive 4 performance points. Our complete version of the explicit free list allocator has a performance index between just over 0.7 and 0.8; it would receive a full 5 points. **Thus if you have a performance index GREATER THAN 0.7 (mdriver prints this as "70/100") then you will get the full 5 points for Performance.**
  - Observing that both memory and CPU cycles are expensive system resources, we adopt this formula to encourage balanced optimization of both memory utilization and throughput. Ideally, the performance index will reach `P = 1` or 100%. To receive a good performance score, you must achieve a balance between utilization and throughput.

- **Synthesis Questions (6 points)**

Resources

Practice Problems

We are providing some extra homework-style practice problems for memory allocation in case you find them helpful in preparing for Lab 5. You do not need to submit these, they are just good practice. Read section 9.9 from the textbook for review. (Note "word" means 4 bytes for these problems)

1. Practice Problem 9.6, p. 849
2. Practice Problem 9.7, p. 852
3. Practice Problem 9.10, p. 864

4. Homework Problem 9.15, p. 879
5. Homework Problem 9.16, p. 879

Heap Simulator

The heap simulator (https://courses.cs.washington.edu/courses/cse351/heapsim/) is a helpful tool to get you familiar with the operations of the heap. Beyond allocating and freeing blocks, you can also export your simulation and save it as a trace file. You can then run your Lab 5 solution with these custom trace files as described in the debugging section below.

Tips and Hints

Getting Started

- Read these instructions.
- Read over the provided code.
- Take notes while doing the above.
- Draw some diagrams of what the data structures should look like before and after various operations.

Debugging

- Use the `mdriver -f` option. During initial development, using tiny trace files will simplify debugging and testing. We have included two such trace files (`short1-bal.rep` and `short2-bal.rep`) that you can use for initial debugging.
- Use the `mdriver -v` and `-V` options. The `-v` option will give you a detailed summary for each trace file. The `-V` will also indicate when each trace file is read, which will help you isolate errors.
- Compile with `gcc -g` and use `gdb` . The `-g` flag tells `gcc` to include debugging symbols, so `gdb` can follow the source code as it steps through the executable. The `Makefile` should already be set up to do this. A debugger will help you isolate and identify out of bounds memory references. You can specify any command line arguments for `mdriver` after the `run` command in `gdb` (*e.g.*, `run -f short1-bal.rep`).
- Keep in mind that the last "block" in the heap is a special marker we will call the **heap footer** that is allocated and has size `0` . The heap footer's tags need to be maintained as well.
- Understand every line of the malloc implementation in the textbook. The textbook has a detailed example of a simple allocator based on an implicit free list. Use this as a point of departure. Don't start working on your allocator until you understand everything about the simple implicit list allocator.
- Write a function that treats the heap as an implicit list, and prints all header information from all the blocks in the heap. Using `fprintf` to print to `stderr` is helpful here because standard error is not buffered so you will get output from your print statements even if the next statement crashes your program.

- Encapsulate your pointer arithmetic in the provided C static inline functions. Pointer arithmetic in memory managers is confusing and error-prone because of all the casting that is necessary. We have supplied static inline functions that do this: see `UNSCALED_POINTER_ADD` and `UNSCALED_POINTER_SUB`.
- Use a profiler. You may find the `gprof` tool helpful for optimizing performance. (`man gprof` or searching online for `gprof` documentation will get you the basics.) If you use `gprof`, see the hint about debugging above for how to pass extra arguments to GCC in the `Makefile`.
- Start early! It is possible to write an efficient malloc package with a few pages of code. However, we can guarantee that it will be some of the most difficult and sophisticated code you have written so far in your career. So start early, and good luck!

Heap Consistency Checker

This is an optional, but recommended, addition that will help you check to see if your allocator is doing what it should (or figure out what it's doing wrong if not). Dynamic memory allocators are notoriously tricky beasts to program correctly and efficiently. They are difficult to program correctly because they involve a lot of untyped pointer manipulation. In addition to the usual debugging techniques, you may find it helpful to write a heap checker that scans the heap and checks it for consistency.

Some examples of what a heap checker might check are:

- Is every block in the free list marked as free?
- Are there any contiguous free blocks that somehow escaped coalescing?
- Is every free block actually in the free list?
- Do the pointers in the free list point to valid free blocks?
- Do any allocated blocks overlap?
- Do the pointers in a heap block point to valid heap addresses?

Your heap checker will consist of the function `int mm_check(void)` in `mm.c`. Feel free to rename it, break it into several functions, and call it wherever you want. It should check any invariants or consistency conditions you consider prudent. It returns a nonzero value if and only if your heap is consistent. This is not required, but may prove useful. When you submit `mm.c`, make sure to remove any calls to `mm_check` as they will slow down your throughput.

Extra Credit

`realloc`

Implement a final memory allocation-related function, `mm_realloc`, in `mm-realloc.c`. The signature for this function, which you will find in your `mm.h` file, is:

```
extern void* mm_realloc(void* ptr, size_t size);
```

The function body is in your `mm-realloc.c` file:

```
void* mm_realloc(void* ptr, size_t size) {
  // ... implementation here ...
}
```

To receive credit, you should follow the contract of the C library's `realloc` exactly (pretending that `malloc` and `free` are `mm_malloc` and `mm_free`, etc.). The man page entry for `realloc` says:

```
The realloc() function changes the size of the memory block pointed to by
ptr to size bytes.  The contents will be unchanged in the range from the
start of the region up to the minimum of the old and new sizes.  If the
new size is larger than the old size, the added memory will not be
initialized.  If ptr is NULL, then the call is equivalent to
malloc(size), for all values of size; if size is equal to zero, and ptr
is not NULL, then the call is equivalent to free(ptr).  Unless ptr is
NULL, it must have been returned by an earlier call to malloc(), calloc()
or realloc().  If the area pointed to was moved, a free(ptr) is done.
```

A good test would be to compare the behavior of your `mm_realloc` to that of `realloc`, checking each of the above cases. Your implementation of `mm_realloc` should also be performant. Avoid copying memory if possible, making use of nearby free blocks. You should not use `memcpy` to copy memory; instead, copy `WORD_SIZE` bytes at a time to the new destination while iterating over the existing data.

To run tracefiles that test `mm_realloc`, compile using `make mdriver-realloc`. Then, run `mdriver-realloc` with the `-f` flag to specify a tracefile, or first edit `config.h` to include additional realloc tracefiles (`realloc-bal.rep` and `realloc2-bal.rep`) in the default list.

Don't forget to submit your finished `mm-realloc.c` along with `mm.c` to the extra credit assignment.

Garbage Collection

Do NOT spend time on this part until you have finished and turned in the core assignment.

You will implement a basic mark-and-sweep garbage collector. Write your implementation in `mm-gc.c`.

Some additional notes:

- To test the current garbage collector, use `make mdriver-garbage` which generates an executable called `mdriver-garbage`.
- The driver assumes that you have a correctly working `mm_malloc` and `mm_free` implementation.
- The tester checks that all of the blocks that should have been freed are freed and that all of the others remain allocated. On success it prints "Success! The garbage collector passed all of the tests." You can look in `GarbageCollectorDriver.c` to see what the test code does.
- This implementation assumes that the alignment is 8 bytes because the third bit of `size_and_tags` is used as the mark bit to mark the block.
- The function `is_pointer` only looks for pointers that point to the beginning of a payload (like Java), and will return false if it points to a free block.
- Pointers will always be word aligned in the data block.

Don't forget to submit your finished `mm-gc.c` along with `mm.c` to the extra credit assignment.

Lab 5 Synthesis Questions

Go back to `part_5` of `lab0.c` and add a `malloc` statement with 16 as its argument *before* the `malloc` for `class_grades`. Recompile the file and examine the difference in the addresses of the two heap blocks in `part_5`. Play around with the size of the new malloc-ed block and answer the following questions (don't assume the implementation is from the first part of the lab):

1. In this implementation, how many bytes in an allocated block are taken up by the boundary tag(s)? Do *not* make any assumptions about the allocator (*e.g.*, the presence of both a header and footer, 8-byte headers)! Briefly describe how you came to this conclusion. [4 pts]

⚠ The following question is open-ended! It is less about "right" or "wrong" and more about you giving us your personal reflection.

2. Interfaces are meant to help abstract away complex detail. After having implemented `malloc` and `free`, are there any benefits you gained by learning these complex details? If so, give an example of an implementation detail in Lab 5 that might help a user use it more performantly. If not, explain why you feel this way. [2 pts]

Submission

You will submit: `mm.c` , and `lab5synthesis.txt` .

After submitting, please wait until the autograder is done running and double-check that you passed the "File Check" and "Compilation and Execution Issues" tests. If either test returns a score of -1, be sure to read the output and fix any problems before resubmitting. Failure to do so will result in a programming score of ZERO for the lab.

Submit your files to the "Lab 5" assignment on Gradescope (../submit.php). Don't forget to add your partner, if you have one.
If you completed any extra credit, also submit `mm.c` , `mm-realloc.c` (if you did `mm-realloc`), and `mm-gc.c` (if you implemented the garbage collector) files to the "Lab 5 Extra Credit" assignment.

It is fine to submit multiple times up until the deadline; we will only grade your last submission. If you do re-submit, you must re-submit BOTH files again AND add your partner again to the new submission.

PAUL G. ALLEN SCHOOL OF COMPUTER SCIENCE & ENGINEERING (https://cs.uw.edu) UW Site Use Agreement (//www.washington.edu/online/terms/)